PEDIDOS DE AMIZADE

Como MySQL, PHP e JavaScript trabalham em conjunto no Arcadia

Este guia acompanha o percurso de um pedido: desde o clique em “Adicionar amigo”, passando pela API e pela base de dados, até à aceitação ou recusa na interface.

Baseado nos ficheiros atuais: database/schema.sql, includes/bootstrap.php, api/friends.php, assets/js/app.js e index.php.

Visão geral

JavaScript regista ações e redesenha	→	PHP / API autentica, valida e grava	→	MySQL guarda relação e avisos

1. O modelo de dados no MySQL

A tabela friendships representa pedidos pendentes e amizades aceites. Cada par de utilizadores tem no máximo uma linha: o estado e os campos direcionais indicam em que fase está a relação.

Campos principais

Campo	Função
user_one, user_two	Par ordenado numericamente (menor ID primeiro). A chave primária composta impede duplicar o par, mesmo se a ordem do pedido for invertida.
requester_id	Utilizador que iniciou o pedido.
recipient_id	Utilizador que recebeu o pedido.
status	pending: à espera de resposta. accepted: amizade confirmada.
created_at / updated_at	Data de criação e última alteração. updated_at atualiza automaticamente.

Exemplo: se a pessoa 14 pede amizade à pessoa 39, guardamos user_one=14, user_two=39, requester_id=14, recipient_id=39 e status=pending. Ao aceitar, basta mudar status para accepted; os campos direcionais preservam quem enviou o primeiro pedido.

Há índices por recipient_id/status e requester_id/status para localizar rapidamente pedidos recebidos e enviados. As chaves estrangeiras ligam os participantes à tabela users e eliminam relações se uma conta for apagada (ON DELETE CASCADE).

A tabela notifications guarda quem recebe o aviso, quem o provocou, o tipo (friend_request ou friend_accepted), o texto, o estado de leitura e a data.

2. O PHP: API e regras de negócio

O ficheiro api/friends.php recebe pedidos HTTP e devolve JSON. Abre a ligação PDO, exige uma sessão autenticada e verifica o método. As consultas preparadas passam valores como parâmetros em vez de concatená-los ao SQL.

Leituras via GET

Sem parâmetros, devolve amigos aceites, pedidos recebidos e pedidos enviados. ?q=... pesquisa por nome ou ID público e inclui o estado da relação; ?suggested=1 lista sugestões sem relação anterior; ?profile=ID devolve perfil, jogos e coleções apenas quando a amizade já foi aceite.

Alterações via POST

As ações POST exigem sessão, token CSRF válido, um user_id inteiro e diferente do próprio utilizador. A API normaliza o par por min/max e então aplica a regra da ação:

Ação	O que acontece
send	Cria uma relação pending e avisa o destinatário. Se já houver pedido pendente inverso, aceita-o diretamente. Relação já existente resulta em conflito.
accept	Confirma que o utilizador atual é o destinatário, muda para accepted, marca o pedido como lido e avisa quem pediu.
decline	Apaga o pedido recebido pendente e marca a notificação como lida.
cancel	Apaga o pedido ainda pendente enviado pelo próprio utilizador e atualiza a notificação.
remove	Apaga a relação aceite entre os dois utilizadores.

Mudanças relacionadas são agrupadas em transações: PHP inicia, atualiza amizade/notificação e confirma; perante erro, faz rollback. A API responde com códigos HTTP, como 401 (sem sessão), 403 (sem permissão), 404 (relação inexistente), 409 (conflito) e 422 (dados inválidos).

3. O JavaScript: interação e atualização

Em assets/js/app.js, o estado em memória mantém listas de amigos e pedidos, resultados da pesquisa e separador selecionado. A função request() usa fetch(), interpreta JSON e converte respostas de erro em mensagens para a interface.

Ao abrir Amigos, o JS carrega listas e sugestões. A pesquisa aguarda 180 ms após a última tecla, exige pelo menos dois caracteres e codifica o termo na URL. Um contador de pesquisas ignora respostas antigas que cheguem atrasadas. Conteúdo textual é escapado antes de inserir HTML.

Os cartões mostram as ações permitidas conforme o estado. Um listener de clique identifica data-friend-action. Recusar, cancelar e remover pedem confirmação. Depois o JS envia POST com action, user_id e token CSRF; quando termina, atualiza listas, sugestões, resultados e notificações sem recarregar toda a página.

4. Percurso de um pedido

1. Encontrar

A pesquisa um nome/ID ou escolhe uma sugestão; o browser faz GET à API.

2. Enviar

O clique em “Adicionar amigo” produz POST com action=send, o ID de B e o token CSRF.

3. Validar

PHP exige sessão, valida CSRF, verifica o ID, impede adicionar a própria conta e confirma que B existe.

4. Gravar

A API ordena os IDs do par, verifica se já existe relação e cria status=pending. Na mesma transação cria uma notificação para B.

5. Atualizar

A resposta JSON chega ao JS. A interface carrega os dados recentes e mostra “Pedido enviado”; B verá o pedido recebido e o contador.

6. Responder

Ao aceitar, status muda para accepted e A é notificado. Ao recusar, a linha pending é apagada. A pode cancelar enquanto continua pendente.

7. Usar a amizade

Ambos aparecem na lista de amigos. Perfil partilhado e chat são acessíveis após aceitação.

Resumo técnico

```
Browser: POST (ação, utilizador, CSRF) → PHP: autenticação e validação → MySQL: verificar/criar/alterar friendships → MySQL: atualizar notifications → PHP: JSON → JS: atualizar estado e interface.
```

5. Segurança e resumo

Autenticação: operações exigem sessão. CSRF: token ligado à sessão acompanha POST. SQL injection: PDO com parâmetros preparados. Autorização: só destinatário aceita/recusa; apenas remetente cancela; perfil exige amizade aceite. Integridade: chave composta com IDs ordenados evita relações duplicadas. Consistência: transações agrupam a relação e os avisos.

MySQL guarda o estado persistente. PHP decide se uma transição é permitida e grava-a. JavaScript apresenta os dados e envia ações; não é ele quem decide se alguém é amigo, porque o servidor valida cada operação.

Referências consultadas: Arcadia/database/schema.sql; Arcadia/includes/bootstrap.php; Arcadia/api/friends.php; Arcadia/assets/js/app.js; Arcadia/index.php. Documento descritivo da implementação encontrada, sem alterações ao código da aplicação.